



AIAA 3111 - Introduction to Data Mining

Lec-4: Pattern Mining: Advanced Methods

Dr. Zhidan Liu



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Lecture 3 Recap: What We Mine

Concept	Question answered	Key condition
Frequent itemset	Which items co-occur?	support $\geq$ minsup
Association rule	Given X, how often does Y follow?	confidence and lift
Closed pattern	Can support be preserved after extension?	no equal-support superset
Maximal pattern	Can the pattern remain frequent after extension?	no frequent superset

Support measures *prevalence*; confidence is *conditional*; lift compares the rule with *independence*.

Pattern Mining: Advanced Methods

- ❑ Mining Multi-level and multi-dimension Patterns
- ❑ Constraint-based Pattern Mining
- ❑ Mining Sequential Patterns
- ❑ Mining Subgraph Patterns
- ❑ Summary

Lecture 4 Roadmap

Part 1: Broader Patterns and Constraints

- ❑ Multi-level and multi-dimensional associations
- ❑ Rare and negatively correlated patterns
- ❑ Constraint properties and pruning strategies

Part 2: Ordered and Structured Patterns

- ❑ Sequential patterns and GSP
- ❑ Pattern growth with projected databases
- ❑ Frequent and closed subgraph mining

Lecture 3 focused on frequent itemsets;
Lecture 4 expands the pattern language and search space.

Why **Advanced** Pattern Mining?

Basic market-basket assumptions

- ❑ One *flat* item universe
- ❑ One support threshold
- ❑ Unordered transactions
- ❑ Mine all qualifying patterns

Real data

- ❑ Items belong to *hierarchies* and *dimensions*
- ❑ *Rare events* can carry high value
- ❑ Order and graph structure may matter
- ❑ Users often specify *constraints*

Advanced methods change the **pattern representation**, the **interestingness measure**, or the **search strategy**.

Mining Multiple-Level Frequent Patterns

- Items often form hierarchies

- Milk: Dairyland 2% milk
- Bread: Wonder wheat bread

Uniform support

Level 1
min_sup = 5%

Level 2
min_sup = 5%

Milk
[support = 10%]

2% Milk
[support = 6%]

Skim Milk
[support = 2%]

Reduced support

Level 1
min_sup = 5%

Level 2
min_sup = 1%

- Min-support thresholds for items following concept hierarchies
 - **Uniform** min-support across multiple levels
 - **Level-reduced** min-support: Items at the lower level are expected to have lower support
- Efficient mining: *Shared* multi-level mining
 - Use the **lowest min-support** to pass down the set of candidates

Redundancy Filtering at Mining Multi-Level Associations

- ❑ **Redundancy filtering:** redundant due to “**ancestor**” relationships
 - ❑ Rule (1): milk $\Rightarrow$ wheat bread [support = 8%, confidence = 70%]
 - ❑ Rule (2): 2% milk $\Rightarrow$ wheat bread [support = 2%, confidence = 72%]
 - ❑ Suppose the 2% milk sold is about $\frac{1}{4}$ of milk sold in gallons
 - ❑ Rule (2) is largely explained by Rule (1) and the prevalence of 2% milk.
- ❑ **In general:** A rule is *redundant* if its **support is close** to the “expected” value, according to its “ancestor” rule, and it has a **similar confidence** as its “ancestor”

Q: If rule (1) is an ancestor of rule (2), which one to prune?

Redundancy Across Concept Levels

Ancestor information gives a baseline for the more specific rule.

1. The ancestor rule *milk* $\Rightarrow$ *wheat bread* has support 8% and confidence 70%.
2. Suppose 2% milk accounts for one quarter of all milk sales.
3. The expected support of *2% milk* $\Rightarrow$ *wheat bread* is about $8\% \times 1/4 = 2\%$.
4. Observed support 2% and confidence 72% add little information beyond the ancestor rule.

A descendant rule becomes interesting when it deviates clearly from the expectation inherited from its ancestor.

Customized Min-Supports for Different Kinds of Items

- ❑ We have used the **same min-support threshold** for all the items or item sets to be mined in each association mining
- ❑ In reality, some items (e.g., diamond, watch, ...) are valuable but less frequent
- ❑ It is necessary to have **customized min-support** settings for different kinds of items
- ❑ One Method: Use **group-based “individualized” min-support**
 - ❑ E.g., {diamond, watch}: 0.05%; {bread, milk}: 5%; ...
 - ❑ How to mine such rules efficiently?
 - ❑ Existing scalable mining algorithms can be easily extended to cover such cases

Mining Multi-Dimensional Associations

- ❑ **Single-dimensional** rules (e.g., items are all in “product” dimension)
 - ❑ $\text{buys}(X, \text{“milk”}) \Rightarrow \text{buys}(X, \text{“bread”})$
- ❑ **Multi-dimensional** rules (i.e., items in ≥ 2 dimensions or predicates)
 - ❑ **Inter-dimension** association rules (*no repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{occupation}(X, \text{“student”}) \Rightarrow \text{buys}(X, \text{“coke”})$
 - ❑ **Hybrid-dimension** association rules (*repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{buys}(X, \text{“popcorn”}) \Rightarrow \text{buys}(X, \text{“coke”})$
- ❑ Attributes can be *categorical* or *numerical*
 - ❑ Categorical Attributes (e.g., *profession*, *product*: no ordering among values): Data cube for inter-dimension association
 - ❑ Quantitative Attributes: Numeric, implicit ordering among values—discretization, clustering, and gradient approaches

Three Forms of Association Rules

Rule type	Predicates	Example
Single-dimensional	One repeated dimension	$\text{buys}(\text{milk}) \Rightarrow \text{buys}(\text{bread})$
Inter-dimensional	No repeated predicate	$\text{age}(18-25) \wedge \text{student} \Rightarrow \text{buys}(\text{coke})$
Hybrid-dimensional	At least one repeated predicate	$\text{age}(18-25) \wedge \text{buys}(\text{popcorn}) \Rightarrow \text{buys}(\text{coke})$

- **Inter-dimensional rules: each predicate appears at most once**
- **Hybrid-dimensional rules: at least one predicate appears more than once**

Rare Patterns vs. Negative Patterns

❑ Rare patterns

- ❑ Very low support but **interesting** (e.g., buying Rolex watches)
- ❑ How to mine them? *Setting individualized, group-based min-support thresholds for different groups of items*

❑ Negative patterns

- ❑ Negatively correlated: Unlikely to happen together
- ❑ Ex.: Since it is unlikely that the same customer buys both a **Ford Expedition** (an SUV car) and a **Ford Fusion** (a hybrid car), buying a **Ford Expedition** and buying a **Ford Fusion** are likely negatively correlated patterns
- ❑ How to define negative patterns?

Defining Negative Correlated Patterns

- A support-based definition
 - If itemsets A and B are both frequent but rarely occur together, i.e.,
 - $\text{sup}(A \cup B) \ll \text{sup}(A) \times \text{sup}(B)$
 - Then A and B are negatively correlated
- Is this a good definition for large transaction datasets?
 - Ex.: Suppose a store sold two needle packages A and B 100 times each, but only one transaction contained both A and B
 - When there are in total 200 transactions, we have
 - $s(A \cup B) = 0.005, s(A) \times s(B) = 0.25, s(A \cup B) \ll s(A) \times s(B)$
 - After adding 19,800 null transactions ($N = 20,000$):
 - $s(A \cup B) = 0.00005, s(A) \times s(B) = 0.000025$, so the conclusion reverses
- What is the problem here?

Does this remind you the definition of *lift*?

Null transactions change the result: this definition is not null-invariant

Defining Negative Correlation: Need **Null-Invariance** in Definition

- A useful negative-correlation measure should be **null-invariant**
 - Intuition: Whether two itemsets A and B are negatively correlated should not be influenced by the number of null-transactions
- A Kulczynski measure-based definition
 - If itemsets A and B are frequent but $(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 < \epsilon$ then A and B are negatively correlated
- For the same needle package problem:
 - The value is unchanged for $N = 200$ and $N = 20,000$
 - If $\epsilon = 0.02$, we have
$$\text{Kulc}(A,B) = (0.01 + 0.01)/2 = 0.01 < \epsilon$$

ϵ is a negative
pattern threshold

Quick Check: Which Measure Is Null-Invariant?

Question: We add 10,000 transactions that contain neither A nor B. Which measure remains unchanged?

A. Relative Support(A)	B. Lift(A,B)
C. Kulc(A,B)	D. Relative Support(AUB)

Answer: C. Kulc(A,B)

Why Constraint-Based Mining?

- ❑ Pattern mining in practice: Often a **user-guided, interactive process**
 - ❑ User directs what to be mined using a **data mining query language** (or a graphical user interface), **specifying various kinds of constraints**
- ❑ **Constraint-based mining: Mine together with user-provided constraints**
- ❑ Why constraint-based mining?
 - ❑ **User flexibility**: User provides constraints on what to be mined
 - ❑ **Optimization**: System explores such constraints for mining efficiency
 - ❑ E.g., Push constraints deeply into the mining process

Constraint-Based Mining Workflow

The same constraint can guide both the result and the search.

1. The user specifies the target data, pattern form, and interestingness conditions.
2. The system converts the query into constraints on patterns and data.
3. The mining algorithm pushes safe constraints into candidate generation or projection.
4. The system verifies the remaining candidates and returns only qualifying patterns.

A constraint improves efficiency only when the system can enforce it before the final output stage.

User Constraints in Pattern Mining

Constraint type	Example	Controls
Data	NY stores this year	Which records enter mining
Dimension / level	brand and customer category	Which attributes appear
Interestingness	support ≥ 0.02	Which patterns qualify
Pattern form	contains milk; length ≤ 4	What the pattern must look like

This lecture focuses on pattern constraints because their mathematical properties enable pruning.

Pattern Space Pruning with **Pattern Anti-Monotonicity**

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is **anti-monotone**

- If an itemset S violates constraint c , so does any of its superset

- That is, mining on itemset S can be terminated

- Ex. 1: $c_1: \text{sum}(S.\text{price}) \leq v$ is **anti-monotone**

- Ex. 2: $c_2: \text{range}(S.\text{profit}) \leq 15$ is **anti-monotone**

- Itemset ab violates c_2 ($\text{range}(ab) = 40$)

$$\text{range}(S.\text{profit}) = \max(S.\text{profit}) - \min(S.\text{profit})$$

- So does every superset of ab

- Ex. 3. $c_3: \text{sum}(S.\text{Price}) \geq v$ is **not anti-monotone**

- Ex. 4. Is $c_4: \text{support}(S) \geq \sigma$ anti-monotone?

- Yes! Apriori pruning is essentially pruning with an anti-monotonic constraint!

Note: item.price > 0
Profit can be negative

Apriori for Pattern Anti-Monotone Constraint

Item	Price
1	1
2	2
3	3
4	4
5	5

Database D

TID	Items
10	1 3 4
20	2 3 5
30	1 2 3 5
40	2 5

Scan D

C_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

F_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{5}	3

Can be chopped early

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

Scan D

C_2

itemset
{1 2}
{1 3}
{1 5}
{2 3}
{2 5}
{3 5}

F_2

itemset	sup
{1 3}	2
{2 3}	2
{2 5}	3
{3 5}	2



C_3

itemset
{2 3 5}

Scan D

F_3

itemset	sup
{2 3 5}	2

Min_sup=2

Constraint:

Sum{S.price} < 5

Quick Check: Anti-Monotone or Not?

Question: Assume all prices are positive. Which constraint is anti-monotone?

A. $\text{sum}(\text{price}) \geq 100$	B. $\text{average}(\text{profit}) \geq 20$
C. $\text{sum}(\text{price}) \leq 100$	D. $\text{length} \geq 3$

Answer: **C.** $\text{sum}(\text{price}) \leq 100$

Pattern Monotonicity and Its Roles

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is *monotone*: If an itemset S satisfies the constraint c , so does any of its superset
 - That is, we do not need to check c in subsequent mining
 - Not as beneficial as anti-monotone
- E.g. 1: $c_1: \text{sum}(S.\text{Price}) \geq v$ is monotone
- E.g. 2: $c_2: \text{min}(S.\text{Price}) \leq v$ is monotone
- E.g. 3: $c_3: \text{range}(S.\text{profit}) \geq 15$ is monotone
 - Itemset ab satisfies c_3
 - So does every superset of ab

Monotone vs. Anti-Monotone Constraints

Property	Anti-monotone	Monotone
Inheritance	Failure passes to supersets	Satisfaction passes to supersets
Search use	Prune after failure	Stop rechecking after satisfaction
Positive prices	$\text{sum}(\text{price}) \leq v$	$\text{sum}(\text{price}) \geq v$
Support	$\text{support} \geq \sigma$	$\text{support} \leq \sigma$

Anti-monotone constraints usually provide stronger early pruning in prefix-growth search.

Convertible Constraints: Ordering Data in Transactions

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

- Convert tough constraints into (anti-)monotone by proper ordering of items in transactions
- Examine c_1 : $\text{avg}(S.\text{profit}) > 20$
 - Order items in (profit) *value-descending order*
 - $\langle a, g, h, b, f, d, c, e \rangle$
 - An itemset ab violates c_1 ($\text{avg}(ab) = 20$)
 - So does ab^* (i.e., *ab-projected DB*)
 - Then c_1 is **not globally anti-monotone**, but it is **convertible anti-monotone under descending-profit order**.

Convertible Constraints Depend on **Growth Order**

- ❑ Average profit is neither monotone nor anti-monotone under arbitrary set extension
- ❑ Sort items by decreasing profit and grow only order-respecting prefixes
- ❑ Once a prefix average falls below the threshold, later lower-profit items cannot restore it
- ❑ This pruning is valid for prefix growth, but not for arbitrary Apriori subset generation

The ordering rule is part of the proof. Without it, the pruning decision is unsafe.

Can item-reordering work for Apriori?

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

Min_sup=2; constraint: $\text{avg}(\text{S.profit}) > 20$

Scan D
→

F_1

itemset	sup.
{a}	3
{f}	4
{g}	3
{d}	3
...	...

F_2

itemset	sup.
{af}	3
{fg}	3
{ag}	2
{ad}	2
...	...

profit-descending order:
<a, g, h, b, f, d, c, e>

Chopped too early

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

- $\text{avg}(gf) = 12.5 < 20$, $\text{avg}(af) = 17.5 < 20$, $\text{avg}(ag) = 35 > 20$
- But $\text{avg}(agf) = 21.7 > 20$
- Apriori will not generate “agf” as a candidate

Reordering enables safe prefix pruning, but not Apriori's all-subset pruning.

Data Space Pruning with Data Anti-Monotonicity

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is **data anti-monotone**: In the mining process, if a data entry t cannot satisfy a pattern p under c , t cannot satisfy p 's superset either
 - Data space pruning: Data entry t can be pruned
- Ex. 1: $c_1: \text{sum}(S.\text{Profit}) \geq v$ is **data anti-monotone**
 - Let constraint c_1 be: $\text{sum}(S.\text{Profit}) \geq 25$
 - $T_{30}: \{b, c, d, f, g\}$ can be removed since none of their combinations can make an S whose sum of the profit is ≥ 25
- Ex. 2: $c_2: \text{min}(S.\text{Price}) \leq v$ is **data anti-monotone**
 - Consider $v = 5$ but every item in a transaction, say T_{50} , has a price higher than 10
- Ex. 3: $c_3: \text{range}(S.\text{Profit}) > 25$ is **data anti-monotone**

Data Space Pruning Should Be Explored **Recursively**

TID	Transaction	Item	Profit
10	a, b, c, d, f, h	a	40
20	b, c, d, f, g, h	b	0
30	b, c, d, f, g	c	-20
40	a, c, e, f, g	d	-15
min_sup = 2		e	-30
		f	-10
		g	20
		h	5

- Example. $c_3: \text{range}(S.\text{Profit}) > 25$
 - We check b's projected database
 - But item "a" is infrequent ($\text{sup} = 1$)
 - After removing "a (40)" from T_{10}
 - T_{10} cannot satisfy c_3 any more
 - Since "b (0)" and "c (-20), d (-15), f (-10), h (5)"
 - By removing T_{10} , we can also prune "h" in T_{20}

b's-proj. DB



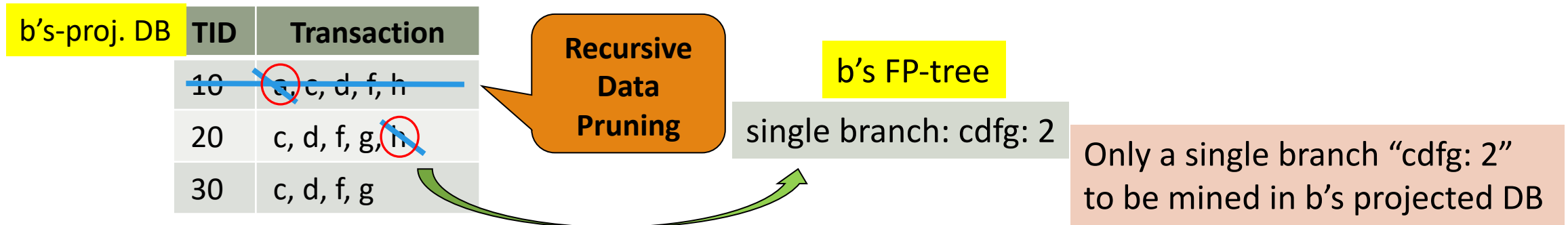
TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

b's-proj. DB

TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

Data Space Pruning Explored **Recursively** (Continued)

Constraint:
 $\text{range}\{S.\text{profit}\} > 25$



- Note: c_3 prunes T_{10} effectively only after "a" is pruned (by min-sup) in b's projected DB

Pattern-Space and Data-Space Pruning

Question	Pattern-space pruning	Data-space pruning
What is removed?	Candidate patterns or branches	Transactions or projected records
Typical reason	No extension can satisfy c	The record cannot support a satisfying extension
Example	support below minsup	maximum attainable profit below v
Benefit	Fewer candidates	Cheaper support counting

A strong mining strategy often reduces both spaces, but each pruning step needs its own **safety argument**.

Succinctness: Pruning Both Data and Pattern Spaces

- **Succinctness:** If the constraint c can be enforced by directly manipulating the data
- E.g. 1: To find those patterns containing item i
 - Mine only i -projected DB (*data space pruning*)
- E.g. 2: To find those patterns without item i
 - Remove i from DB and then mine (*pattern space pruning*)
- E.g. 3: $c_3: \min(S.Price) \leq v$ is succinct
 - Start from qualifying low-price items and mine their projected databases; high-price items may still join them
- E.g. 4: $c_4: \sum(S.Price) \geq v$ is not succinct
 - It cannot be determined beforehand since sum of the price of itemset S keeps increasing

Succinct Constraints

Can be enforced before mining

- ❑ contains item i
- ❑ excludes item i
- ❑ $\min(\text{price}) \leq v$, using qualifying anchors and projected data

Cannot be decided in advance

- ❑ $\text{sum}(\text{price}) \geq v$
- ❑ $\text{average}(\text{profit}) \geq v$ without a valid order
- ❑ $\text{support} \geq \sigma$ before counting occurrences

Succinctness asks *whether direct data manipulation can enforce the constraint exactly.*

Constrained FP-Growth: Push a Succinct Constraint Deep

Item	Price
1	1
2	2
3	3
4	4
5	5

TID	Items
10	1 3 4
20	2 3 5
30	1 2 3 5
40	2 5

Remove
infrequent
length 1

TID	Items
10	1 3
20	2 3 5
30	1 2 3 5
40	2 5

Min_sup=2

Constraint: $\min\{S.\text{price}\} \leq 2$

No Need to project on 3 or 5

1-Projected DB

TID	Items
10	3
30	2 3 5

2-Projected DB

TID	Items
20	3 5
30	1 3 5
40	5

Different Kinds of Constraints Lead to Different Pruning Strategies

- In summary, constraints can be categorized as **pattern space pruning** constraints vs. **data space pruning** constraints

Pattern space pruning constraints	Data space pruning constraints
■ Anti-monotonic: If constraint c is violated, its further mining can be terminated ■ Monotonic: If c is satisfied, no need to check c again ■ Convertible: c can be converted to monotonic or anti-monotonic if items can be properly ordered in processing ■ Succinct: If the constraint c can be enforced by directly manipulating the data	■ Data succinct: Data space can be pruned at the initial pattern mining process ■ Data anti-monotonic: If a transaction t does not satisfy c, then t can be pruned to reduce data processing effort

How to Handle Multiple Constraints?

- It is beneficial to use multiple constraints in pattern mining
- But different constraints may require potentially *conflicting* item-ordering
 - If there exists conflict ordering between c_1 and c_2
 - Try to sort data and enforce *one constraint* first (which one?)
 - Then enforce the other constraint when mining the projected databases
- E.g. c_1 : $\text{avg}(S.\text{profit}) > 20$, and c_2 : $\text{avg}(S.\text{price}) < 50$
 - Assume c_1 has more pruning power
 - Sort in profit descending order and use c_1 first
 - For each projected DB, sort items by increasing price and enforce c_2 during mining

Handling Multiple Constraints

- ❑ **Estimate** which constraint removes the largest part of the search space
- ❑ **Apply** a succinct or strongly anti-monotone constraint as early as possible
- ❑ **Preserve** the item ordering required by a convertible constraint
- ❑ **Reorder** within projected databases only when the later pruning proof remains valid
- ❑ **Verify** all constraints before output, even when some were used for early pruning

The best enforcement order depends on pruning power, evaluation cost, and ordering compatibility.

Part 1 Synthesis: **What Can We Prune?**

Situation	Safe action	Reason
Pattern fails anti-monotone c	Stop growing the branch	All supersets fail
Pattern satisfies monotone c	Stop checking c on supersets	All supersets satisfy
Constraint is succinct	Filter or project data first	Direct enforcement is exact
Record cannot satisfy any extension	Remove the record	Data anti-monotonicity

Part 2 changes the pattern representation from unordered sets to *sequences and graphs*.

Appendix

Backup Slides for Part 1

Lecture 3 Recap: Apriori

Stage	Operation	Reason
1. Join	Generate C_k from compatible pairs in F_{k-1}	Only frequent $(k-1)$ -itemsets can contribute
2. Prune	Remove a candidate if any $(k-1)$ -subset is absent from F_{k-1}	An infrequent subset makes every superset infrequent
3. Count	Scan the transaction database for the surviving candidates	Support must be measured on the data
4. Select	Keep X when $\text{support}(X) \geq \text{minsup}$ to form F_k	Repeat until F_k is empty

Apriori alternates candidate generation and database scans; downward closure provides the pruning guarantee.

Lecture 3 Recap: Apriori Optimizations and Alternatives

Method	What changes	Efficiency idea
DHP	Apriori candidate filtering	Hash k-itemsets and prune low-count buckets; reduce transactions
Partition	Apriori database access	Mine local frequent sets, then verify their union in a second scan
ECLAT	Vertical TID-list representation	Intersect TID sets in a depth-first search
FP-growth	Compressed prefix-tree representation	Mine conditional pattern bases without explicit candidates

DHP and Partition optimize Apriori; ECLAT and FP-growth reorganize the data and the search.

Part 1 Learning Goals

Broader pattern spaces require stronger control over what we mine.

- Choose support thresholds for hierarchical and heterogeneous items
- Distinguish rare patterns from negatively correlated patterns
- Classify constraints as anti-monotone, monotone, convertible, or succinct
- Decide whether a constraint can prune patterns, data, or both

Uniform vs. Level-Reduced Support

Policy	Example	Strength	Risk
Uniform minsup	5% at every level	Simple and comparable	Misses specific low-level items
Level-reduced minsup	5% for dairy; 1% for milk types	Finds detailed patterns	Produces more candidates

Lower-level items divide the parent support, so they often need a lower threshold.

Rare Patterns and Negative Patterns

Property	Rare pattern	Negative pattern
Main question	Does the pattern occur infrequently?	Do A and B co-occur less than expected?
Required marginals	Not central	A and B should each occur often enough
Typical measure	Support with a customized threshold	A null-invariant correlation measure
Example	Rolex purchase	Two substitute products

Low joint support alone cannot distinguish rarity from negative association.

Various Kinds of User-Specified Constraints in Data Mining

- ❑ **Knowledge type constraint** — Specifying what kinds of knowledge to mine
 - ❑ E.g.: Classification, association, clustering, outlier finding, ...
- ❑ **Data constraint** — using SQL-like queries
 - ❑ E.g.: Find products sold together in NY stores this year
- ❑ **Dimension/level constraint** — similar to projection in relational database
 - ❑ E.g.: In relevance to region, price, brand, customer category
- ❑ **Interestingness constraint** — various kinds of thresholds
 - ❑ E.g.: Strong rules: $\text{min_sup} \geq 0.02$, $\text{min_conf} \geq 0.6$, $\text{min_correlation} \geq 0.7$
- ❑ **Rule (or pattern) constraint**  **The focus of this study**
 - ❑ E.g.: Small sales (price < \$10) triggers big sales (sum > \$200)

Kulczynski Measure Is Null-Invariant

Kulc averages the two conditional co-occurrence probabilities.

1. Use $\text{count}(A)=100$, $\text{count}(B)=100$, and $\text{count}(A \cup B)=1$.
2. $P(B|A)=1/100=0.01$ and $P(A|B)=1/100=0.01$.
3. $\text{Kulc}(A,B)=[P(B|A)+P(A|B)]/2=(0.01+0.01)/2=0.01$.
4. Adding null transactions changes N but does not change these conditional probabilities.

With a negative-correlation threshold $\epsilon = 0.02$, $\text{Kulc} = 0.01$ indicates a negative association.

Constraint Properties and Their Uses

Property	Safe inference	Main use
Anti-monotone	failure persists under extension	prune pattern branches
Monotone	satisfaction persists under extension	stop repeated checking
Convertible	ordering creates one of the above properties	enable prefix pruning
Succinct	direct selection enforces the constraint	reduce the initial search space
Data anti-monotone	a record cannot support later extensions	prune projected data

A constraint may have more than one useful property.

Worked Strategy for Three Constraints

Constraint order is an algorithm-design decision, not only a query-language detail.

1. Requirement: $\text{support} \geq 2$. Keep Apriori or FP-growth support pruning active throughout.
2. Requirement: $\text{min}(\text{price}) \leq 50$. Start from qualifying low-price anchors and projected databases.
3. Requirement: $\text{average}(\text{profit}) > 20$. Use a validated profit-descending growth order where possible.
4. Count and verify the surviving patterns against all three constraints before output.

**Enforce cheap, safe, and high-pruning constraints first;
postpone expensive checks that cannot prune safely.**

Part 1 Check

Question: A pattern violates an anti-monotone constraint. What can the miner safely do?

A. Keep all supersets	B. Prune every superset
C. Declare it maximal	D. Lower minsup

Answer: B. Prune every superset

Part 2: Sequential and Graph Patterns

Order first, then structure

Pattern Mining: Advanced Methods

- ☐ Mining Multi-level and multi-dimension Patterns
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns
- ☐ Summary

Sequential Pattern Mining

- ❑ What kind of patterns are sequential?
- ❑ Sequential: The order really matters
 - ❑ You cannot swap two items in a sequence and have the same sequence
- ❑ Example: The English language is sequential: Subject → Verb → Object

- ❑ Other points:
 - ❑ Sequential patterns use event order; exact timestamps and time gaps are optional constraints
 - ❑ Time-series analysis models measurements indexed by time and often uses the actual intervals

Itemsets and Sequences Ask Different Questions

Representation	Equivalent?	Example question
Itemset {A,B}	$\{A,B\} = \{B,A\}$	Which items occur together?
Sequence $\langle A,B \rangle$	$\langle A,B \rangle \neq \langle B,A \rangle$	Which event tends to happen first?
Event set $\langle\langle A\ B \rangle\rangle$	A and B share one event	Which items occur at the same step?

Sequential pattern mining preserves order between events but not order inside one event set.

Sequential Pattern Examples

- ❑ Application of Sequential pattern Mining
 - ❑ **Customer shopping:** Purchase a laptop first, then a digital camera, and then a smartphone
 - ❑ **Medical treatments:** Go to see a doctor, get drugs, doctor monitors progress, doctor reacts accordingly → more/less drugs
 - ❑ **Natural disasters:** Before the disaster, during the disaster, after the disaster
 - ❑ **Scientific Experiments:** Step 1, Step 2, Step 3
 - ❑ **Market events:** rise, correction, and recovery may form a recurring ordered pattern
 - ❑ **Biological sequences:** changing nucleotide or amino-acid order changes the resulting sequence and function

Reading Sequence Notation

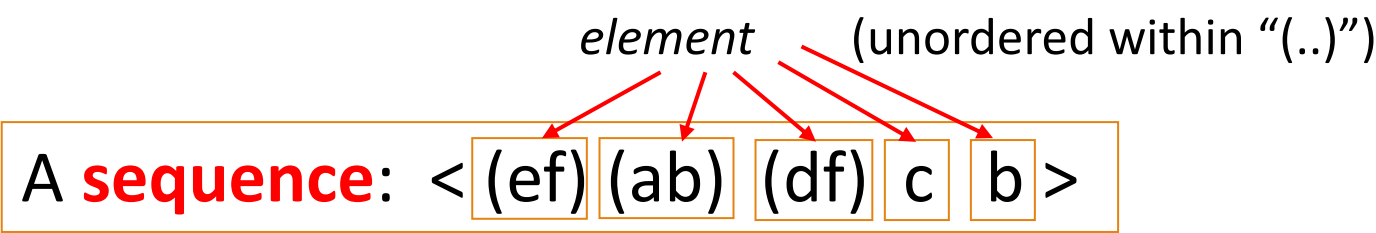
Parentheses group items observed in the same event.

1. Read $\langle a (bc) d \rangle$ from left to right as three ordered elements.
2. The first element contains item a.
3. The second element contains b and c together; their internal order does not matter.
4. The third element contains d.

Sequence length usually counts items, while sequence size counts elements; state which convention the algorithm uses.

Sequential Pattern and Sequential Pattern Mining

- Sequential pattern mining: Given a set of sequences, find the **complete set of frequent subsequences** (i.e., satisfying the min_sup threshold)



- An element may contain a set of items (also called events)

* Items within an element are **unordered** and we list them alphabetically

A **sequence database**

SID	Sequence
10	<a(<u>ab</u> c)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

How a Subsequence Matches

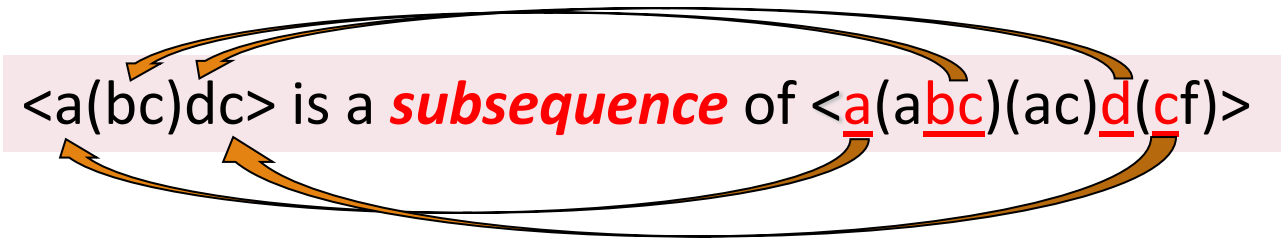
Matching uses ordered containment, not string equality.

1. Target pattern: $\langle a \ (bc) \ d \ c \rangle$.
2. Database sequence: $\langle a \ (abc) \ (ac) \ d \ (cf) \rangle$.
3. Match a to the first element and (bc) to the second element because $\{b,c\} \subseteq \{a,b,c\}$.
4. Match d to the fourth element and c to the fifth element; unused elements may be skipped.

A subsequence preserves order but does not require adjacent elements or exact element equality.

Sequential Pattern and Sequential Pattern Mining

- **Sequential pattern mining:** Given a set of sequences, find the **complete set of frequent subsequences** (i.e., satisfying the min_sup threshold)



- Given support threshold $min_sup = 2$, <(ab)c> is a **sequential pattern**

A **sequence database**

SID	Sequence
10	<a(<u>ab</u> c)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

Support of a Sequential Pattern

- A sequence supports pattern s when s is a subsequence of that sequence
- Each sequence ID contributes at most one support count, even if s appears many times
- $\text{support count}(s) = \text{number of sequence IDs containing } s$
- $\text{relative support}(s) = \text{support count}(s) / \text{number of sequences}$

Frequency counts supporting sequences, not the number of embeddings inside them.

Sequential Pattern Mining Algorithms

- ❑ Algorithm requirement: Efficient, scalable, finding complete set, incorporating various kinds of user-specific constraints
- ❑ The Apriori property still holds: If a subsequence s_1 is infrequent, none of s_1 's super-sequences can be frequent
- ❑ Representative algorithms
 - ❑ GSP (Generalized Sequential Patterns): Srikant & Agrawal, EDBT '96

GSP: Level-Wise Sequential Mining

The Apriori property also holds for subsequences.

1. Find frequent length-1 sequences.
2. Join compatible frequent length-k sequences to generate length-(k+1) candidates.
3. Prune candidates whose required subsequences are infrequent.
4. Scan the sequence database to count support and retain frequent candidates.
5. Repeat until no candidates remain.

GSP transfers Apriori pruning from itemsets to ordered subsequences, but still requires repeated candidate generation and scans.

GSP: Apriori-Based Sequential Pattern Mining

- Initial candidates: All 8-singleton sequences
 - $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle, \langle e \rangle, \langle f \rangle, \langle g \rangle, \langle h \rangle$
- Scan DB once, count support for each candidate

SID	Sequence
10	$\langle (bd)cb(ac) \rangle$
20	$\langle (bf)(ce)b(fg) \rangle$
30	$\langle (ah)(bf)abf \rangle$
40	$\langle (be)(ce)d \rangle$
50	$\langle a(bd)bcb(ade) \rangle$

$min_sup = 2$

Cand.	sup
$\langle a \rangle$	3
$\langle b \rangle$	5
$\langle c \rangle$	4
$\langle d \rangle$	3
$\langle e \rangle$	3
$\langle f \rangle$	2
$\langle g \rangle$	1
$\langle h \rangle$	1



GSP (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

Three Types of Length-2 Candidates

Candidate form	Meaning	Count with m frequent items
$\langle a\ b \rangle$	a occurs before b	m^2 , including repeated items
$\langle b\ a \rangle$	b occurs before a	already included in m^2
$\langle (a\ b) \rangle$	a and b occur together	$m(m-1)/2$

With $m=6$, GSP generates $6^2 + 6 \times 5 / 2 = 51$ length-2 candidates.

GSP: Apriori-Based Sequential Pattern Mining

- Example: Generate length-2 candidate sequences

singleton * singleton – Total: (6 * 6)

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

min_sup = 2

Cand.	sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1



Sets (unordered) – Total: (6*5) / 2

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Apriori Pruning

- w/o pruning
(includes g and h):

$$8*8 + 8*7/2 = 92$$

length-2 candidates

- w/ pruning:

$$6*6 + 6*5/2 = 51$$

length-2 candidates

GSP Mining and Pruning

			<u>length</u>
5 th scan: 1 cand. 1 length-5 seq. pat.	<(bd)cba>	☐ Remove Candidates not in DB or Candidates < min_sup	5
4 th scan: 8 cand. 7 length-4 seq. pat.	<abba> <(bd)bc> ...		4
3 rd scan: 46 cand. 20 length-3 seq. pat. 20 cand. not in DB at all	<abb> <aab> <aba> <baa> <bab> ...		3
2 nd scan: 51 cand. 19 length-2 seq. pat. 10 cand. not in DB at all	<aa> <ab> ... <af> <ba> <bb> ... <ff> <(ab)> ... <(ef)>		2
1 st scan: 8 cand. 6 length-1 seq. pat.	<a> <c> <d> <e> <f> <g> <h>		1

$$6*6 + 6*5/2 = 51$$

The GSP algorithm

- ☐ Repeat, starting at k = 1 until k <= length
 - ☐ Scan DB to find “length-k” frequent sequences
 - ☐ Generate “length-(k+1)” candidate sequences from “length-k” frequent sequences using **Apriori**
 - ☐ set k = k+1
- ☐ Until no frequent sequence or no candidate can be found

min_sup = 2

SID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

Why GSP Becomes Expensive

Computation

- ❑ One database scan per sequence length
- ❑ Large candidate sets at early levels
- ❑ Costly subsequence matching during counting

Search waste

- ❑ Many candidates never occur in the database
- ❑ Long patterns create many overlapping candidates
- ❑ Sparse sequences provide little candidate reuse

Pattern-growth methods avoid global candidate generation by working inside projected databases.

PrefixSpan: Grow Patterns by Prefix

- Choose a frequent prefix, such as $\langle a \rangle$
- Build the prefix-projected database containing only suffixes after an occurrence of the prefix
- Count locally frequent extension items in that smaller database
- Extend the prefix and recurse without generating a global candidate set

Projection converts one large search problem into smaller conditional problems.

A Simple Prefix-Projected Database

SID	Original sequence	Suffix after prefix ⟨a⟩
S1	⟨a b c⟩	⟨b c⟩
S2	⟨a c b⟩	⟨c b⟩
S3	⟨b a c⟩	⟨c⟩

Within the ⟨a⟩-projected database, b appears in two suffixes and c appears in three.

Growing Prefixes in PrefixSpan

Each recursive call mines only sequences compatible with its prefix.

1. Start with prefix $\langle a \rangle$ and its projected database.
2. If minsup count is 2, both b and c qualify as extensions.
3. Create $\langle a \ b \rangle$ and $\langle a \ c \rangle$, then build a new projected database for each prefix.
4. Continue until no locally frequent extension remains.

PrefixSpan still relies on support anti-monotonicity; it changes how the search space is represented and explored.

GSP and PrefixSpan

Aspect	GSP	PrefixSpan
Search	breadth-first	depth-first pattern growth
Candidates	explicit global candidates	prefix extensions
Database access	repeated full scans	recursive projected databases
Main cost	candidate generation and matching	projection size and recursion

Neither method dominates in every dataset; sequence density and pattern length shape the trade-off.

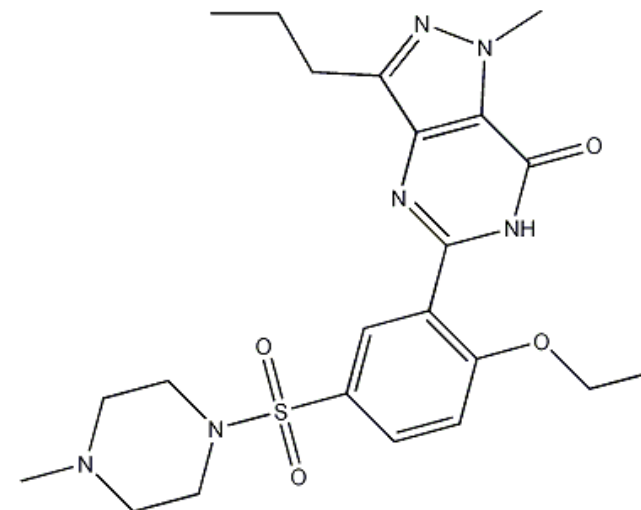
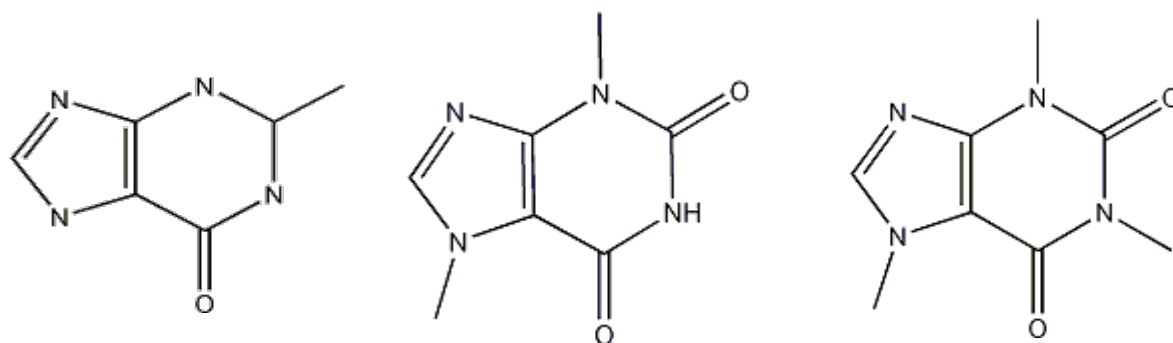
Pattern Mining: Advanced Methods

- ❑ Mining Multi-level and multi-dimension Patterns
- ❑ Constraint-based Pattern Mining
- ❑ Mining Sequential Patterns
- ❑ Mining Subgraph Patterns 
- ❑ Summary

What Is Graph Pattern Mining?

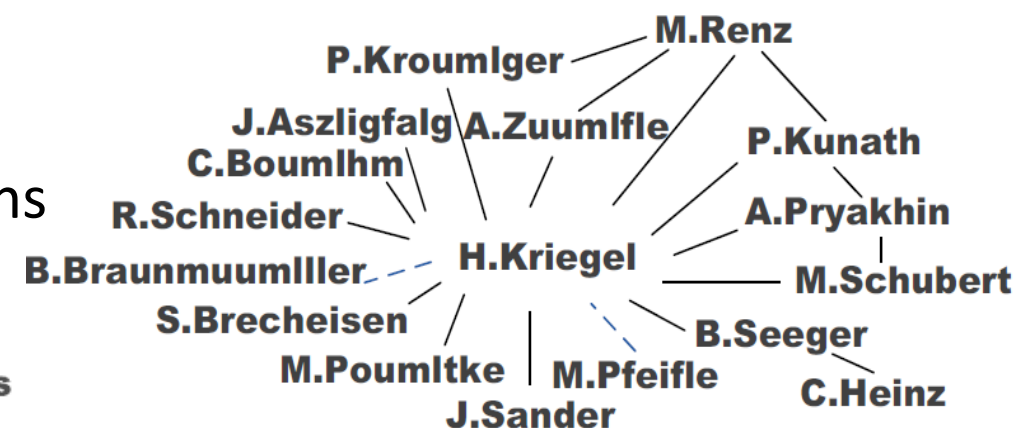
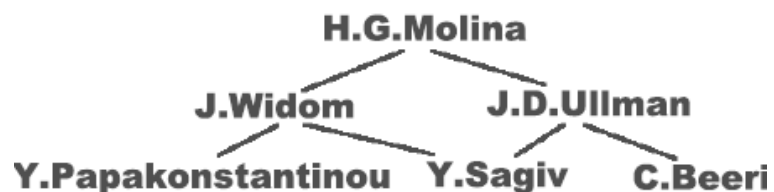
- Chem-informatics:

- Mining frequent chemical compound structures



- Social networks, web communities, tweets, ...

- Finding frequent research collaboration subgraphs



Labeled Graph Patterns

Term	Meaning	Example
Vertex label	type of an entity	carbon atom; user role
Edge label	type of a relation	chemical bond; follows
Subgraph	selected vertices and edges preserving labels	functional group
Embedding	one mapping of the pattern into a graph	one occurrence location

Graph pattern mining must handle both combinatorial growth and graph isomorphism.

Frequent (Sub)Graph Patterns

- Given a labeled graph dataset $D = \{G_1, G_2, \dots, G_n\}$, the supporting graph set of a subgraph g is $D_g = \{G_i \mid g \subseteq G_i, G_i \in D\}$

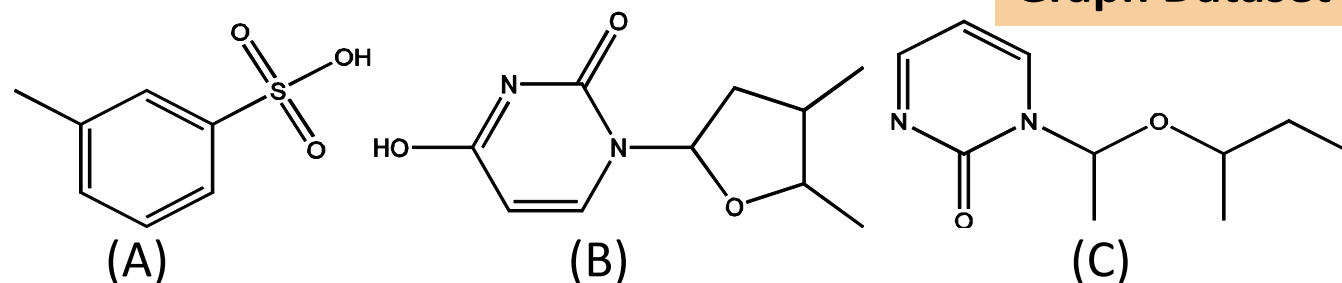
$\text{support}(g) = |D_g| / |D|$

- A (sub)graph g is **frequent** if $\text{support}(g) \geq \text{min_sup}$

- Ex.: Chemical structures

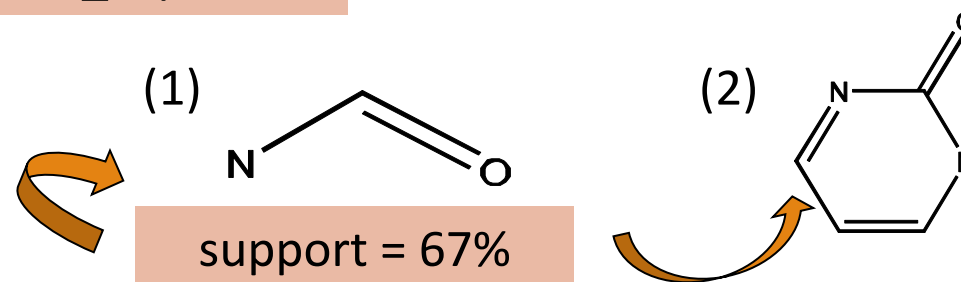
- Alternative:

- Mining frequent subgraph patterns from a single large graph or network



min_sup = 2

Frequent Graph Patterns



Support Is Counted Across Graphs

Do not confuse embedding count with graph-level support.

1. Let D contain three labeled graphs G_1 , G_2 , and G_3 .
2. Pattern g occurs twice in G_1 , once in G_2 , and not in G_3 .
3. The supporting graph set is $D_g = \{G_1, G_2\}$.
4. $\text{support count}(g) = 2$ and $\text{relative support}(g) = 2/3$.

Multiple embeddings inside one graph still contribute one supporting graph in a graph-transaction database.

Why Support Counting Is Hard for Graphs

For itemsets

- Containment is a simple subset test
- Canonical item order is immediate
- Support counting can use hashing or set intersection

For graphs

- Containment requires a label-preserving node mapping
- One graph has many equivalent drawings and traversals
- Subgraph isomorphism dominates the counting cost

A graph miner needs both pruning and a canonical representation.

Quick Check: Graph Support

A subgraph appears five times in G1, once in G2, and zero times in G3. What is its support count?

A. 1	B. 2
C. 5	D. 6

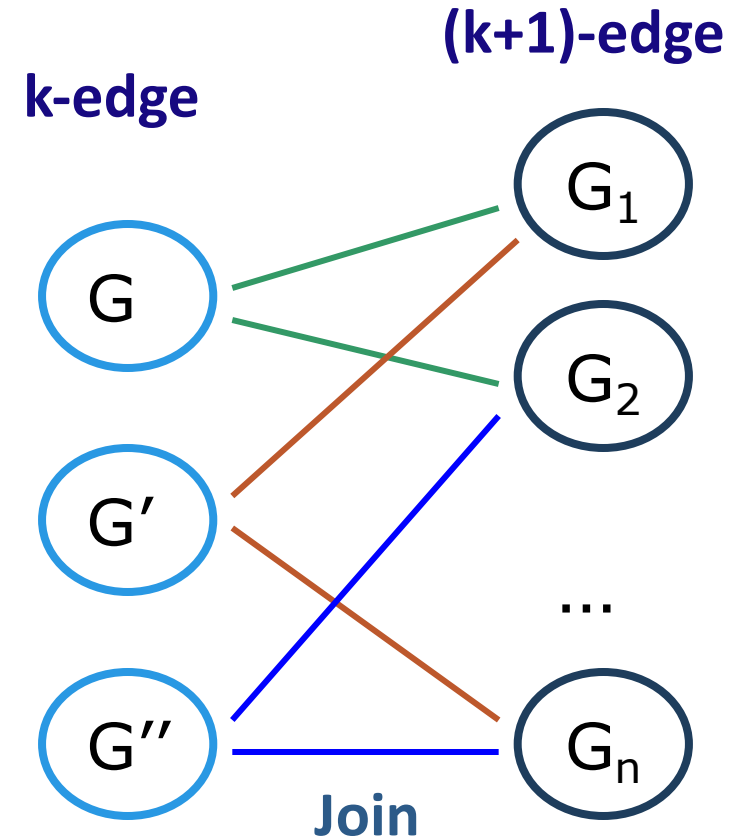
Answer: B. 2 supporting graphs

Applications of Graph Pattern Mining

- ❑ Bioinformatics
 - ❑ Gene networks, protein interactions, metabolic pathways
- ❑ Chem-informatics: Mining chemical compound structures
- ❑ Social networks, web communities, tweets, ...
- ❑ Cell phone networks, computer networks, ...
- ❑ Web graphs, XML structures, Semantic Web, information networks
- ❑ Software engineering: Program execution flow analysis
- ❑ Building blocks for graph classification, clustering, compression, comparison, and correlation analysis
- ❑ Graph indexing and graph similarity search

Apriori-Based Approach

- Apriori property: if a size- k subgraph is frequent, every subgraph is frequent; one infrequent subgraph prunes the candidate
- Generate a size- $(k+1)$ candidate by joining compatible frequent size- k subgraphs
- Iterative mining process:
 - Candidate-generation $\rightarrow$ candidate pruning $\rightarrow$ support counting $\rightarrow$ candidate elimination



Apriori-Style Graph Mining

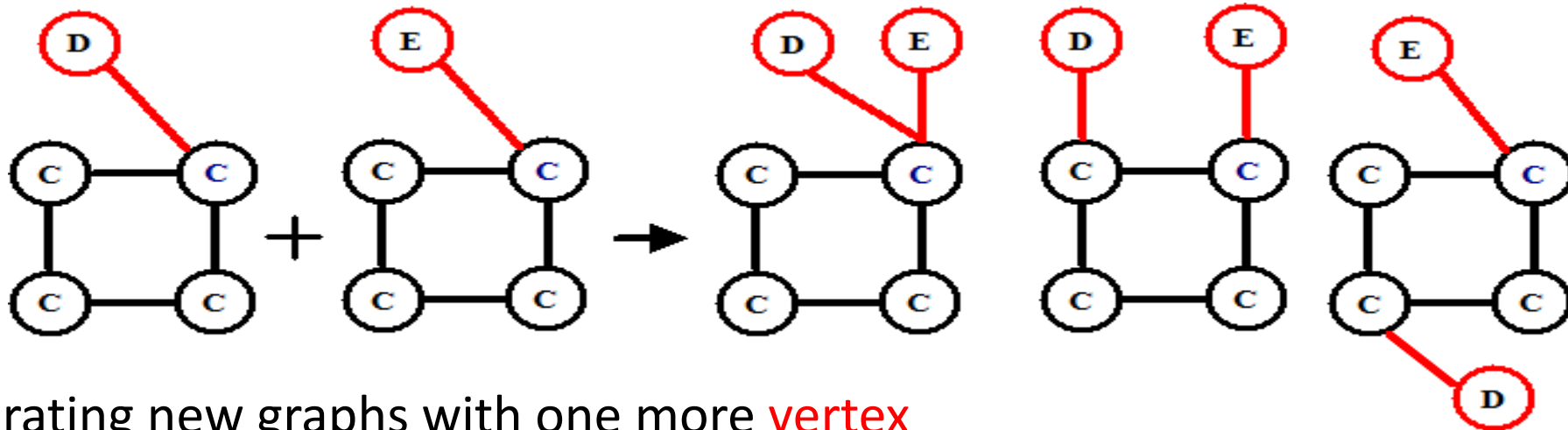
Graph joins are more complicated than itemset joins because the shared subgraph can have many mappings.

1. Join compatible frequent size- k subgraphs to create size- $(k+1)$ candidates.
2. Canonicalize candidates and remove duplicates.
3. Prune a candidate if any required size- k subgraph is infrequent.
4. Count support by testing which database graphs contain the candidate.

Frequent implies that every subgraph is frequent; the converse does not hold.

Candidate Generation: Vertex Growing vs. Edge Growing

- ❑ Methodology: breadth-first search with Apriori-style joins of size- k graphs
 - ❑ Many possibilities at generating size- $(k+1)$ candidate graphs



- ❑ Generating new graphs with one more **vertex**
 - ❑ AGM (Inokuchi, Washio, & Motoda, PKDD'00)
- ❑ Generating new graphs with one more **edge**
 - ❑ FSG (Kuramochi & Karypis, ICDM'01)
- ❑ Edge growing often reduces candidate duplication compared with vertex growing

The Duplicate-Candidate Problem

- The same unlabeled structure can be drawn with different vertex numbers
- Different growth histories can produce isomorphic candidates
- Naive deduplication requires repeated graph-isomorphism tests
- A canonical code assigns one representative to every graph isomorphism class

gSpan avoids most duplicates by growing only canonical DFS codes.

Pattern-Growth Approach

- Depth-first growth of subgraphs from k -edge to $(k+1)$ -edge, then $(k+2)$ -edge subgraphs

- Major challenge

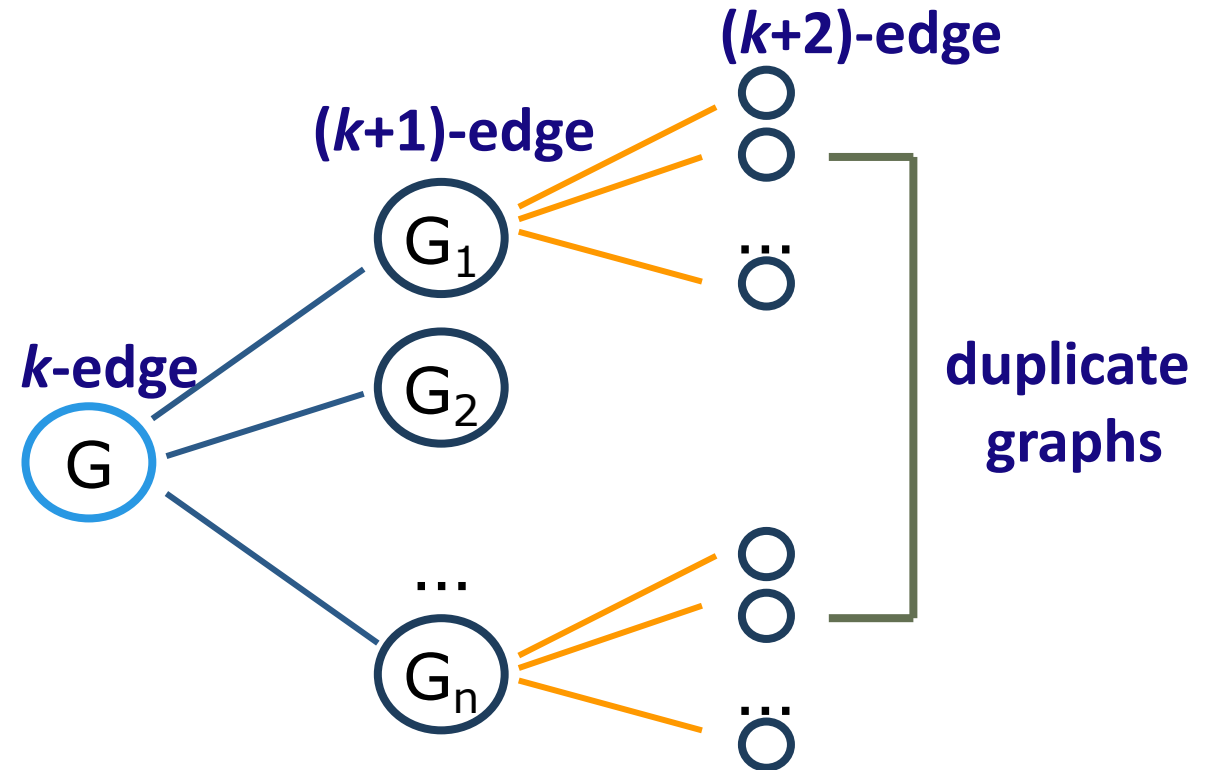
- Generating many duplicate subgraphs

- Major idea to solve the problem

- Define an order to generate subgraphs

- DFS spanning tree: Flatten a graph into a sequence using depth-first search

- gSpan (Yan & Han, ICDM'02)



Canonical DFS Code in gSpan

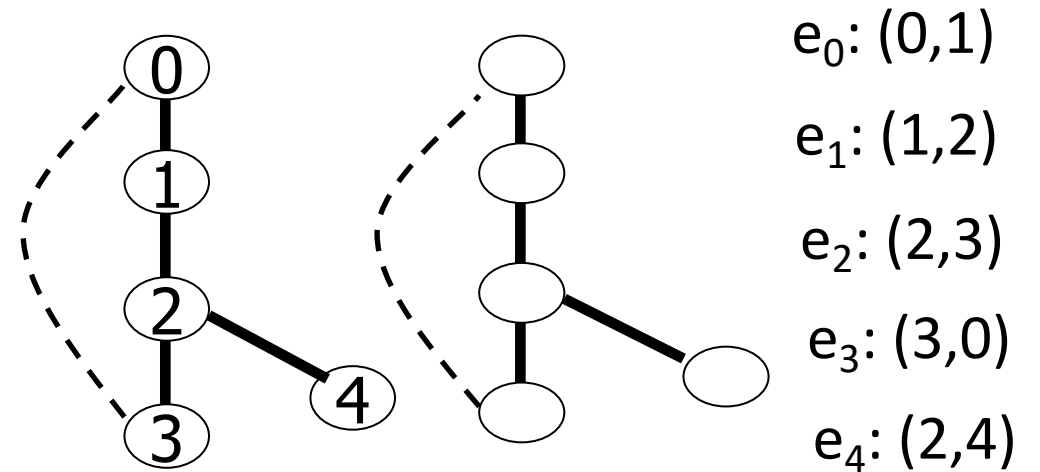
The code represents structure, labels, and traversal order.

1. Run depth-first search and record each traversed edge with endpoint indices and labels.
2. One graph may have many valid DFS codes because traversal can start and branch differently.
3. Compare the codes lexicographically and keep the minimum DFS code.
4. Only the minimum code represents the graph during mining.

Canonical coding turns graph-equivalence checking into a consistent ordering test during growth.

gSPAN: Graph Pattern Growth in Order

- ❑ **Right-most path extension** in subgraph pattern growth
 - ❑ Right-most path: the DFS-tree path from the root to the most recently discovered vertex
 - ❑ Reduce generation of duplicate subgraphs
- ❑ **Completeness:** The enumeration of graphs using right-most path extension is complete
- ❑ DFS code: Flatten a graph into a sequence using depth-first search



Right-Most Path Extension

Forward extension

- Start from a vertex on the current right-most path
- Add a new vertex and one new edge
- The new vertex becomes the latest DFS vertex

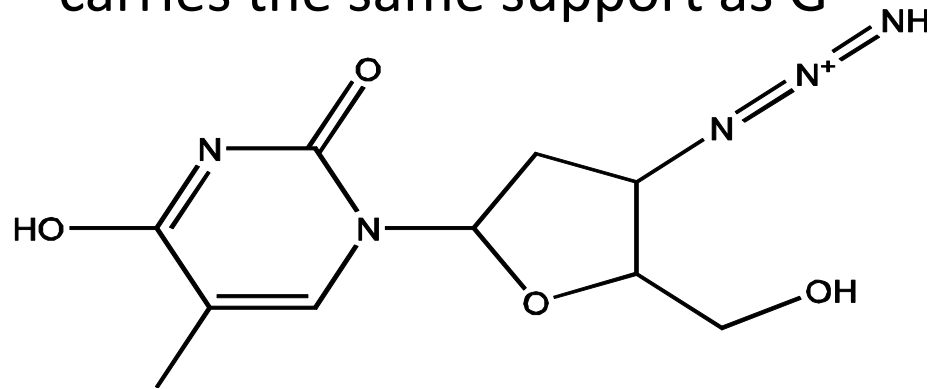
Backward extension

- Start from the current right-most vertex
- Connect to an ancestor on the right-most path
- Close a cycle without adding a vertex

Restricting extensions to the right-most path preserves completeness while controlling duplicates.

Mining Closed Graph Patterns

- Up to 2^n edge subsets $\rightarrow$ closed frequent subgraphs
- A frequent graph G is *closed* if there exists no supergraph of G that carries the same support as G



Closed means that no proper frequent supergraph has the same supporting graph set

- *Lossless compression*: Does not contain non-closed graphs, but still ensures that the mining result is complete
- Algorithm CloseGraph: Mines closed graph patterns directly

Closed Frequent Graphs

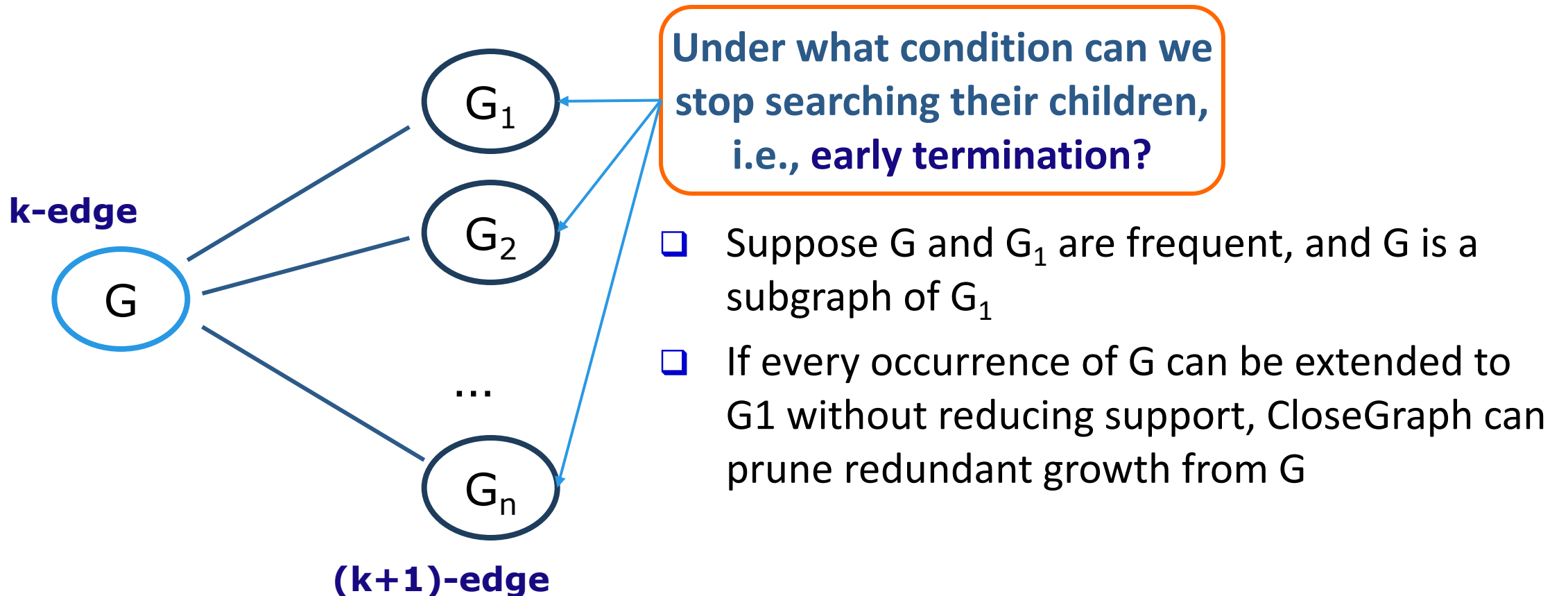
The definition is the same idea as closed itemsets, with graph containment replacing set containment.

1. Graph G is frequent and supported by $\{G_1, G_2, G_3\}$.
2. A proper supergraph H is also supported by $\{G_1, G_2, G_3\}$.
3. G and H therefore have equal support.
4. G is not closed because H adds structure without losing any supporting graph.

Closed graph patterns remove support-equivalent redundancy while preserving exact support information.

CloseGraph: Directly Mining Closed Graph Patterns

- CloseGraph: Mining closed graph patterns by extending gSpan (Yan & Han, KDD'03)



CloseGraph Early Termination

- Suppose every occurrence of G can be extended to a larger graph H
- G and H then share the same supporting graph set
- G cannot be closed because H is a proper equal-support supergraph
- Continue growth from the larger representative and prune redundant descendants of G when the closure condition is safe

The pruning decision depends on occurrence-level equivalence, not only equal numeric support.

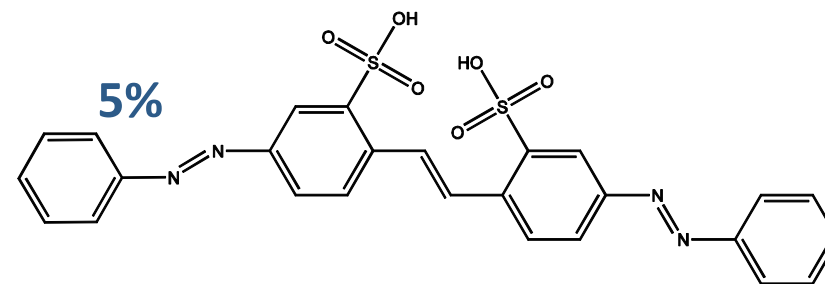
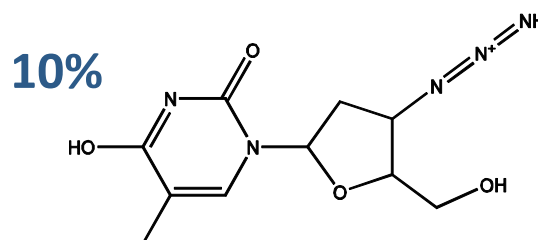
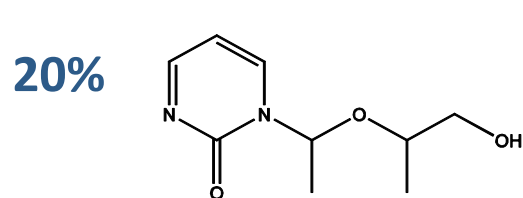
Frequent Graph Mining Algorithms

Method	Search	Duplicate control	Output
AGM / FSG	breadth-first candidates	canonicalize after joins	all frequent graphs
gSpan	depth-first pattern growth	minimum DFS code	all frequent graphs
CloseGraph	gSpan-based closed growth	closure-based pruning	closed frequent graphs

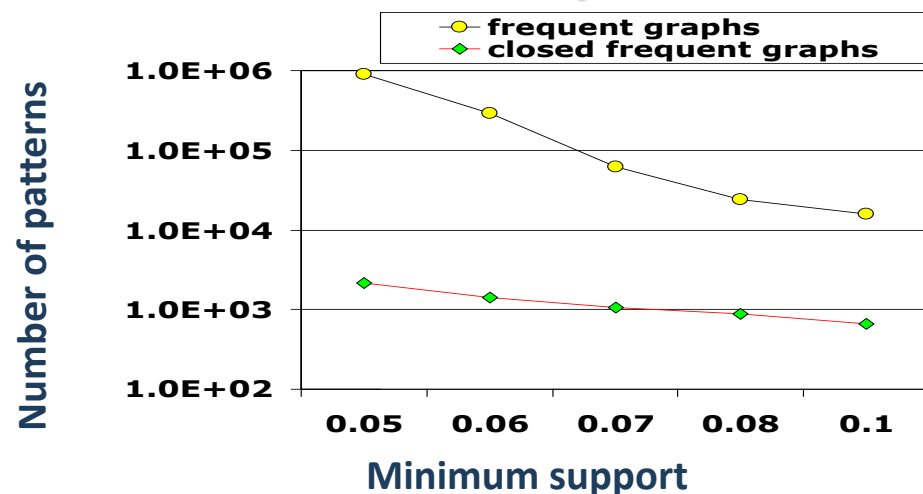
The methods differ mainly in enumeration order, duplicate handling, and output compression.

Experiment and Performance Comparison

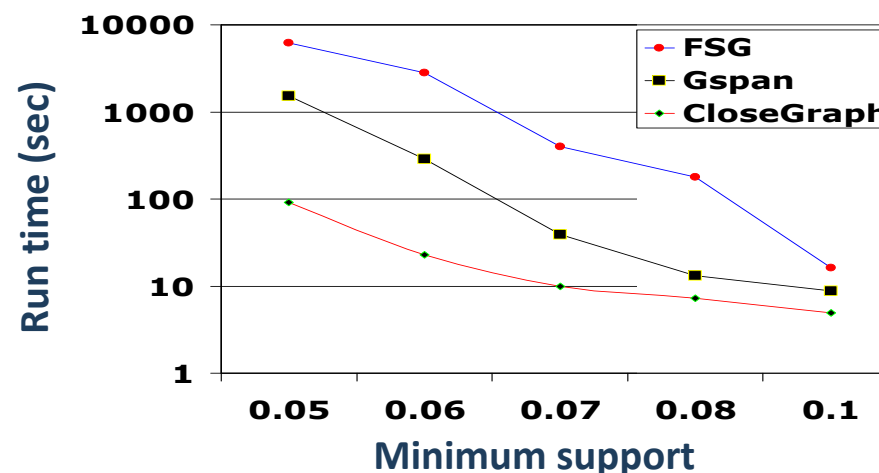
- ❑ The AIDS antiviral screen compound dataset from NCI/NIH
- ❑ The dataset contains 43,905 chemical compounds
- ❑ Discovered patterns: The smaller minimum support, the bigger and more interesting subgraph patterns discovered



of Patterns: Frequent vs. Closed



Runtime: Frequent vs. Closed

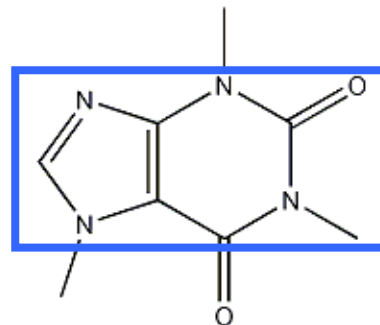
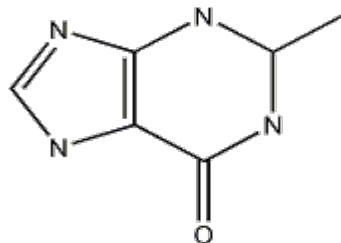


Application: Support Substructure Similarity Search

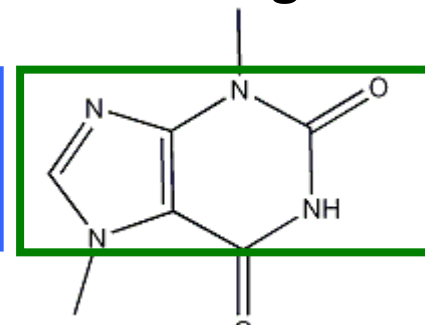
- Find graphs in a graph DB containing substructures similar to a given query graph

- Ex. Data: A chemical compound DB

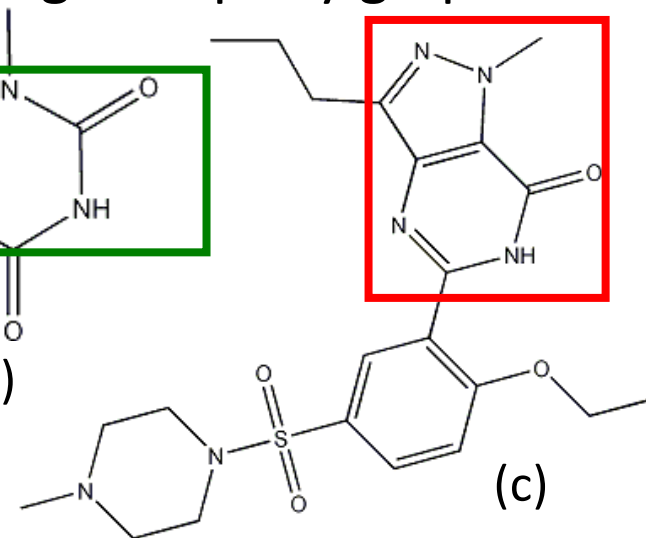
- A query graph q:



(a)



(b)



(c)

- How to do similarity search efficiently?

- Sequential scan + computing subgraph similarity – too costly!

- Build graph indices to support approximate search?

- Need an explosive number of subgraphs to cover all the *similar* subgraphs!

- An elegant solution (Yan, Yu, & Han, SIGMOD'05):

- Keep the graph index structure, but **select features** in the **query space**

Lecture 4 Synthesis

Data and pattern	Main challenge	Representative idea
Hierarchical / multidimensional items	thresholds and redundancy	level-aware support and ancestor filtering
Constraint-based itemsets	large irrelevant search space	push safe constraints into mining
Sequences	order and many candidates	GSP or prefix-projected growth
Graphs	isomorphism and duplicates	canonical DFS growth and closed patterns

Across all four settings, representation and anti-monotonicity determine what can be pruned safely.

Final Quick Check

Which statement is correct?

A. Order never matters in a sequence	B. A candidate with an infrequent subgraph can be pruned
C. Every closed graph is maximal	D. Lift is null-invariant

Answer: B